

Сдача работы и баллы

Работу необходимо будет сдать лично – на паре или в доп. время. Не тяните, распределяйте время, чтобы не было столпотворений!

При проверке будет оцениваться соответствие всем нижеперечисленным требованиям, реализация правильной архитектуры, умение применять используемые технологии, удобство пользования системой, защищенность системы.

Необходимо использовать Docker (сервер приложений и сервер СУБД в отдельных контейнерах). Желательно задеплоить на общедоступный сервер. Код необходимо выложить на Github Classroom в специально подготовленное преподавателем задание.

Работа без выполнения списка обязательных требований ниже не принимается. Невыполнение каждого дополнительного требования к наполнению/реализации уменьшает максимальные баллы на 20%.

Обязательные требования

Семестровая работа – сайт, написанный с использованием Spring MVC Framework. Это должно быть МРА (допускается SPA, если сумеете объяснить принципы и ответить на вопросы)..

У сайта должна быть идея, обладающая новизной и актуальностью. Если сомневаетесь в сложности и новизне проекта, лучше подойдите-уточните. Либо же можно воспользоваться своей предыдущей семестровой и оптимизировать её.

Сайт должен иметь удобный интерфейс.

Необходимо использовать HTML5, CSS3. CSS-препроцессоры не запрещаются, но нужно знать их основные принципы. Если используете CSS-фреймворки (Bootstrap и проч.), должны уметь пояснить, что за классы используете, особенно классы сетки.

Необходимо корректно работать с формами: проверка данных, защита от повторной отправки там, где это нужно.

Нужно использовать любой шаблонизатор, *можно JSP*. В коде видов нельзя напрямую вызывать Java-код. Должна быть развитая система шаблонов (свои теги или наследование шаблонов, разделение видов на несколько частей).

Сайт должен иметь аутентификацию, авторизацию, регистрацию.

Сайт должен быть защищён от атак, которые обсуждались в течение всего курса предмета.

Сайт должен хранить данные в PostgreSQL. Можно ли использовать другие СУБД нужно уточнить у преподавателя.

Для работы с СУБД необходимо использовать и Spring Data JPA, и JPA. То есть должны быть как и репозитории от Spring, так и самостоятельно написанные запросы с помощью JPQL/CriteriaBuilder.

Система должна работать, как минимум с шестью JPA сущностями. Должны быть связи M2M, O2M.

Как минимум, для одной из сущностей должен быть реализован полностью CRUD: не обязательно в виде админки, но должны быть некие экшены контроллеров, которые реализуют CRUD-действия.

Система должна быть построена по модели MVC, и следует придерживаться логики Controller->Service->Repository(DAO) и т.д. Необходимо придерживаться принципов SOLID. Весь код должен быть корректно поделен на пакеты. Рекомендуется использовать DTO-, Request-, Forms-классы, где это уменьшает связность, абстрагирует архитектуру.

Проект должен быть основан на Spring Boot: нужно использовать стартеры, настройки и запускающий класс.

Все реализации требований должны вписываться в бизнес-логику и выполнение соответствующего кода должно быть возможно из-за действий пользователя.

Требования к наполнению, наличию реализаций

- Необходимо применить Javascript. Нужно использовать AJAX.
- Все исключения, проблемы должны корректно обрабатываться: обычные экшены – приводить к отдаче самостоятельно оформленной веб-страницы, а AJAX-запросы получить JSON-ответ с информацией о проблеме. Не должно быть дефолтных страниц с ошибками от сервера приложений или Spring Boot.
- Использование стороннего API, не используя специальную библиотеку для этого API (SDK), а самостоятельное описание

запросов на уровне HTTP (например, библиотека okhttp). Примеры: вход или восстановление пароля через отправку смс, использование курсов валют для пересчёта стоимости корзины. Наличие простого информера для сайта типа вывода погоды не подходит!

- Использование Spring Security. То есть полноценная работа с пользователем: закрытая часть сайта, аутентификация, регистрация.
- Организация REST API хотя бы для одной сущности. Подключение генерации openAPI. А также написание тестов через Postman или аналоги.
- Использование конвертеров, в том числе и своих.
- Логирование возникающих исключений.
- Наличие 2 нестандартных методов для репозитория в Spring: самостоятельно написанных методов с использованием @Query, CriteriaBuilder, но не повторяющих функционал методов, описываемых в Spring Data JPA.
- Один запрос с подзапросом (subselect).
- Наличие кэширования с помощью Redis или аналогов.

Сроки

Дедлайн досрочной сдачи: 9 мая. Можно получить +4 балла баллов сверх баллов семестровой работы при строгом выполнении всех требований.

Дедлайн сдачи: 30 мая.

Разбалловка

При несоответствии каждому дополнительному требованию максимальные баллы уменьшаются на 20%.

Если обнаружено, что какой-то участок кода списан, то баллы списавшего и давшего списать, делятся пополам или аннулируются полностью.

Все уменьшения баллов должны быть целочисленными с округлением в меньшую сторону.

1. **Фронтэнд** (вёрстка, дизайн, шаблоны, JS) – 2
2. **Работа с базой** (репозитории, 4 сущности, M2M, O2M, свои запросы) – 4
3. **Архитектура** (слои абстракций, взаимодействие модулей, шаблоны проектирования) – 5
4. **Общие правила построения сайтов** (упоминание контекста, защита от повторной отправки, проверки ввода, корректный вывод, обработка запроса и проч) – 6
5. **Использование продвинутых техник/технологий**, в т.ч. Spring (логгирование, правильное описание конфигураций, настройка окружения, конвертеров, централизованная обработка исключений и т.д.) – 7
6. **Корректное использование API** – 1

Итого: 25 баллов

Пример: студент сдал работу без 2 дополнительных требований и у него было очень много проблем с фронтэндом. 60% максимальных баллов и вычет всех баллов за фронтэнд: $(1 - 0.2 * 2) * (25 - 3) \text{ балла} = 15 - 1.8 = 13.2 \approx 14 \text{ баллов}$.

Доп.задание

Подключить OAuth-сервис без использование библиотеки, которая непосредственно решает эту задачу. Максимум: библиотека для отправки HTTP-запросов, генерации секретных кодов. Нужно реализовать функционал “войти через”.

+4 балла (но не более 25 суммарно)